

Taller 1: Docker + Login

Sistema de gestión de proyectos de dispositivos médicos de misión crítica

¿Qué es un contenedor?

Un contenedor empaqueta una aplicación junto con todo lo que necesita para correr: su código, sus librerías, la versión exacta del runtime y su configuración. Ese paquete corre igual en cualquier máquina, porque no depende de nada instalado afuera.

- No es una máquina virtual: comparte el kernel del sistema operativo, por eso arranca en milisegundos y pesa megabytes, no gigabytes.
- Una **imagen** es la plantilla inmutable. Un **contenedor** es una instancia viva de esa imagen.
- Varios contenedores se coordinan con `docker compose`: un archivo declara todos los servicios y arrancan juntos, en la misma red.

Qué hay en el repositorio

```
git clone git@github.com:CE-Projects-Avargas/CE-5508-Workshops.git
cd CE-5508-Workshops/01-docker-login
```

Lo que ya está construido y funcionando:

- `docker-compose.yml` — orquesta los tres contenedores del sistema.
- `database/` — MariaDB con el esquema completo ya creado: tablas Usuarios y Proyectos, más datos de ejemplo.
- `backend/` — API REST en Node + Express que expone `/proyectos` (listar y crear). Funciona completo.
- `frontend/` — Aplicación React con dos pantallas: la de creación de proyectos (funcional) y la de login (solo el formulario, sin lógica).
- `auth-service/` — Carpeta vacía. Aquí va su trabajo de hoy.

Importante. La base de datos ya trae la tabla Usuarios lista. No tienen que diseñarla ni crearla — solo usarla.

El reto de hoy

Parte 1 — Instalar Docker y dejar corriendo el código base.

Instalen Docker Desktop, clonen el repositorio y levanten el sistema. Al terminar esta parte deben poder abrir el frontend en el navegador, crear un proyecto y verlo aparecer en la lista.

```
docker compose up --build
```

Parte 2 — Construir un contenedor aparte que resuelva la autenticación.

Levanten un **servidor separado, en su propio contenedor**, independiente del backend de proyectos, que se conecte a la misma base de datos MariaDB y resuelva dos cosas:

- **Registro:** recibir los datos de un usuario nuevo e insertarlo en la tabla `Usuarios`.
- **Login:** recibir credenciales, validarlas contra lo que está guardado, y responder si son correctas o no.

Luego conecten la pantalla de login del frontend a ese servicio nuevo. El objetivo final: **que un login exitoso permita pasar a la pantalla de creación de proyectos**, que ya existe y ya funciona. Sin login válido, no se pasa.

Criterio. Cómo lo resuelvan es decisión de cada grupo: lenguaje, framework, librerías y estructura son suyos. Lo que se evalúa es que funcione, que corra en su propio contenedor, y que puedan explicar cada decisión que tomaron.

Trabajo en grupo

- El taller se resuelve **en grupos**. Cada grupo se organiza como prefiera: repartan el trabajo, definan quién hace qué y cómo integran lo de cada quien.
- Organizarse es parte del ejercicio. En un equipo real nadie les va a asignar las tareas.

Sobre el uso de herramientas

Pueden usar **cualquier herramienta** para resolver el reto: documentación, foros, plantillas, librerías, generadores de código e inteligencia artificial. No hay restricción.

La única condición es que la declaren y la entiendan:

- **Mencionar** qué herramientas usaron.
- **Explicar para qué** las usaron y en qué parte del reto.
- **Saber cómo funciona** lo que entregaron. Si no pueden explicar una línea de su solución, esa línea no cuenta.

Regla clara. Usar IA está permitido y es realista. Entregar código que no pueden explicar, no.